

RESULT SYSTEM

Project Report

ABSTRACT

This report explains how I built a Student Result Processing System using C programming language. The main goal was to create a simple tool that helps teachers calculate student results without doing everything by hand. The system takes input like student roll numbers, names, and marks in five subjects. Then it automatically calculates total marks, percentage, and assigns grades based on standard grading rules.

I used Merge Sort to arrange student records in order of roll numbers and Binary Search to quickly find any student's details. The system also shows a complete result sheet and a leaderboard that displays the top three students. This project helped me understand how sorting and searching algorithms work in real-world applications. The program runs on the command line and is easy to use with a simple menu system.

CHAPTER 1: INTRODUCTION

1.1 Background

Every school and college needs to process student results at the end of each term or semester. Teachers have to calculate marks from different subjects, add them up, find percentages, and assign grades. This is a lot of work when you have many students. I have seen my own teachers spending hours checking marksheets and doing calculations. Sometimes they make mistakes too, which creates problems later.

I thought there must be a better way to do this. Computers are good at doing calculations fast and without errors. So I decided to make a program that can do all this work automatically. This program takes the raw marks and gives complete results in seconds.

1.2 Why I Made This Project

There were several reasons that made me want to build this system:

First, I wanted to save time. When teachers have to calculate results for 50 or 60 students, it takes many hours. My program can do the same work in just a few moments.

Second, I wanted to reduce errors. I have seen cases where a teacher adds marks wrong or calculates percentage incorrectly. This affects students' grades unfairly. A computer program does the same calculation every time without getting tired or making mistakes.

Third, I wanted to make searching easy. If a student wants to check their marks, finding their record from piles of papers is difficult. My program lets you search by roll number and shows

the result immediately.

Fourth, I wanted to try out the algorithms we learned in class. We studied Merge Sort and Binary Search in our data structures course. This project gave me a chance to actually use them in a real program.

1.3 What the System Does

This system is designed to handle student results from start to finish. Here is what it can do:

- Take input for any number of students (up to 100)
- Store their roll numbers, names, and marks in five subjects
- Calculate total marks automatically
- Find percentage for each student
- Assign grades from A to F based on marks
- Sort all students by roll number
- Search for any student quickly
- Show a complete result sheet
- Display the top three students in a leaderboard

The program is made for teachers and school staff who need to process results quickly. It works on any computer that has a C compiler installed.

1.4 What I Wanted to Achieve

My main goals for this project were:

1. Build a working system that actually processes student results
2. Use Merge Sort to organize student data
3. Use Binary Search to find student records fast
4. Make a simple menu that anyone can use
5. Show results in a neat, readable format
6. Learn how theory from class applies to real programs

CHAPTER 2: PROBLEM STATEMENT

2.1 The Real Problem

In most schools and colleges, result processing is still done the old way. Teachers use paper registers or maybe Excel sheets. But both methods have problems.

2.1.1 Takes Too Much Time

When I talked to some teachers, they told me it takes them about two to three days to calculate results for one class of 50 students. First they add marks for each subject. Then they add all subjects to get total. Then they calculate percentage. Then they figure out grades. If they have multiple classes, this takes even longer.

2.1.2 Mistakes Happen Often

Teachers are human. They get tired after checking many answer sheets. In this tired state, they can add numbers wrong. Sometimes they forget to add one subject. Sometimes they calculate percentage incorrectly. These mistakes mean students get wrong grades sometimes.

2.1.3 Finding Old Records is Hard

If a student comes asking about their marks from last year, finding that paper record is like searching for a needle in a haystack. Teachers have to go through piles of registers and marksheets. It wastes a lot of time.

2.1.4 No Ranking System

When schools need to find the top students for prizes or awards, they have to manually compare everyone's marks. This means looking at each student's total and figuring out who is first, second, and third. This is very time-consuming when you have many students.

2.2 What I Decided to Build

I decided to make a program that solves all these problems. My program would:

- **Save time** by doing all calculations instantly
- **Avoid mistakes** by using a computer instead of human calculation
- **Make searching easy** with quick student lookup
- **Show rankings** automatically with a leaderboard feature
- **Display results clearly** in a well-formatted table

The program uses proper algorithms like Merge Sort and Binary Search to work efficiently. Even with many students, it will give results quickly.

2.3 Who Benefits From This

This system helps different people in different ways:

Teachers - They don't have to do manual calculations anymore. They just enter marks once and everything else is done automatically.

Students - They get accurate results without any calculation mistakes. They can also see their rank in the class.

School Administration - They can maintain proper records and give reports to parents quickly.

Parents - They get reliable information about their children's performance.

CHAPTER 3: LITERATURE REVIEW

3.1 What Other Systems Exist

Before I started making my program, I looked at other systems that schools use. I wanted to see what was already available and what I could do better.

3.1.1 Excel Spreadsheets

Many schools use Excel for keeping student marks. Teachers make columns for each subject, then use formulas to add marks and calculate percentages. This works okay for small classes.

But Excel has problems too. If someone accidentally deletes a formula or changes a cell by mistake, it can mess up all calculations. Also, Excel doesn't have a proper search function for student records. You have to scroll through rows to find a student.

3.1.2 School Management Software

Some schools buy expensive software like PowerSchool or Blackbaud. These programs do everything from attendance to fees to results. They are very powerful.

But these programs are costly. Small schools cannot afford them. They also need training to use properly. Many teachers find them too complicated.

3.1.3 Homemade Systems

Some schools use custom software made by local programmers. These are usually better than Excel but not as good as commercial software. The quality varies a lot.

3.2 Comparing Different Algorithms

3.2.1 Sorting Methods

For my project, I needed to sort students by roll number. I looked at different sorting algorithms:

Merge Sort - This algorithm divides the list into halves, sorts each half, and merges them back. It always works in $O(n \log n)$ time no matter what the data looks like. It needs extra space though.

Quick Sort - This is another fast sorting algorithm. It picks a pivot element and arranges smaller elements on one side and larger on the other. It's usually faster than Merge Sort but can be slow in worst case.

Bubble Sort - This is the simplest sorting method. It compares adjacent elements and swaps them if needed. It's very slow for large lists because it takes $O(n^2)$ time.

Selection Sort - This finds the smallest element and puts it at the beginning, then finds the next smallest, and so on. Also $O(n^2)$ time.

I decided to use Merge Sort because:

- It always gives good performance ($O(n \log n)$)
- It doesn't have a bad worst-case scenario
- I wanted to use the algorithm we studied in class
- The extra space requirement was okay for my limited data size

3.2.2 Search Methods

For finding a student by roll number, I looked at two options:

Linear Search - This checks each student one by one until it finds the right one. Simple to implement but slow. If you have 100 students, it might check all 100.

Binary Search - This requires the data to be sorted first. Then it keeps dividing the search space in half. With 100 students, it only needs to check about 7 students maximum.

Binary Search is much faster, so I chose it. But it only works because I already sort the data using Merge Sort.

3.3 What I Learned From Others

Looking at existing systems gave me some ideas for my project:

1. Keep it simple - don't make too many features
2. Make it work on any computer without special software
3. Use clear formatting so results are easy to read
4. Include basic features like search and ranking
5. Make it interactive with a menu

I decided to make my program in C because it's simple, fast, and runs everywhere. I didn't want to use complicated databases or graphical interfaces because that would make it harder to use.

CHAPTER 4: REQUIREMENT ANALYSIS

4.1 What the System Must Do

I thought carefully about what features this system absolutely needs. These are the functional requirements.

4.1.1 Getting Student Data

The system must ask the user how many students they want to enter. Then for each student, it must take:

- Roll number (a number like 101, 102, etc.)
- Name (text like "Rahul" or "Priya")
- Marks for 5 different subjects

The system should store this data properly so it can be used later.

4.1.2 Doing Calculations

After getting all the data, the system must automatically:

- Add marks of all 5 subjects to get total marks
- Calculate percentage by dividing total by 500 and multiplying by 100
- Assign a grade based on the percentage:
 - 90% and above → A grade
 - 75% to 89% → B grade
 - 60% to 74% → C grade
 - 40% to 59% → D grade
 - Below 40% → F grade

4.1.3 Showing Results

The system must show a nice result sheet that displays:

- Roll number
- Name
- Total marks
- Percentage
- Grade

Everything should be in a table format that is easy to read.

4.1.4 Search Feature

The user should be able to search for a student by roll number. If the student exists, the system shows all their details. If not, it says "Student Not Found."

4.1.5 Leaderboard

The system must show the top three students based on total marks. It should display:

- 1st place: Name, total marks, percentage, grade
- 2nd place: Same details
- 3rd place: Same details

4.1.6 Menu System

There should be a menu with options:

1. Display Result Sheet

2. Search Student
3. Leaderboard
4. Exit

The user can choose what they want to do after the calculations are done.

4.2 Other Important Requirements

Besides the main features, there are some other things the system must handle.

4.2.1 Performance

The system should work quickly even with many students. Sorting should not take more than a few seconds. Search should find a student immediately.

4.2.2 Reliability

The system should not crash if the user enters wrong input. It should handle errors gracefully. It should give clear messages so the user knows what to do.

4.2.3 Simplicity

The system should be easy to use. The instructions should be clear. The menu should be straightforward. Anyone with basic computer knowledge should be able to use it.

4.2.4 Portability

The system should work on any computer that has a C compiler. It shouldn't need special libraries or software. Just compile and run.

4.3 What the System Cannot Do

It's also important to know the limits of this system:

- It can only handle up to 100 students (I set this limit to keep it simple)
- It only works for 5 subjects (can't change the number)
- It doesn't save data to a file (all data is lost when program closes)
- It doesn't have a graphical interface (only command line)
- It doesn't allow editing student data after entry

These are limitations I accepted to keep the project manageable. Maybe in the future I can add these features.

CHAPTER 5: SYSTEM DESIGN

5.1 How the System is Organized

I planned the system in layers. Each layer has a specific job. This makes the code easier to write and understand.

5.1.1 Three Layers

User Interface Layer - This is what the user sees. It shows the menu, asks for input, and displays results. All the printf and scanf statements are in this layer.

Business Logic Layer - This is where the actual work happens. Calculating marks, sorting, searching, making leaderboard - all these functions are in this layer.

Data Layer - This is where the data is stored. The Student structure and the arrays s[] and temp[] are in this layer.

5.1.2 Why I Chose This Design

I chose this three-layer design because:

- It separates different responsibilities
- Makes the code easier to test
- If I change something in one layer, it doesn't affect others
- It's easier to understand what each part does

5.2 Different Parts of the System

5.2.1 Input Part

This part handles all the data entry. It:

1. Asks for number of students
2. Loops through each student
3. Takes roll number, name, and marks
4. Stores everything in the s[] array

This is the first thing that happens when the program runs.

5.2.2 Calculation Part

This part does all the math. It:

1. Goes through each student
2. Adds marks from all 5 subjects
3. Calculates percentage
4. Decides the grade

This runs right after input is complete.

5.2.3 Sorting Part

This part arranges students in roll number order. It uses the Merge Sort algorithm. The sorted data is needed for Binary Search to work.

5.2.4 Search Part

This part finds a specific student. It uses Binary Search. You give it a roll number and it gives back the student's details or says not found.

5.2.5 Display Part

This part shows the results. It formats everything neatly in a table with proper headings.

5.2.6 Leaderboard Part

This part finds the top three students. It makes a copy of the data, sorts it by marks, and shows the top three.

5.2.7 Menu Part

This part controls what happens after the initial processing. It shows options and calls the right function based on user choice.

CHAPTER 6: DATA STRUCTURES USED

6.1 The Student Structure

I created a structure called Student to hold all information about one student. Here's what it looks like:

```
c
struct Student
{
    int roll;

    char name[30];

    int marks[5];

    int total;

    float percentage;

    char grade;
};
```

6.1.1 Why I Made It This Way

I decided to use a structure instead of separate arrays because it keeps all information about one student together. When I sort or search, I don't have to worry about keeping different arrays in sync. Everything moves together.

roll - I used int because roll numbers are usually numbers. It's easy to compare and sort.

name[30] - I used a character array of size 30. This is enough for most Indian names. Some names might be longer but 30 is usually safe.

marks[5] - I used an array of 5 integers because students have 5 subjects. Each subject mark is a whole number.

total - This stores the sum of all marks. Maximum is 500.

percentage - This stores the percentage as a float. I needed decimal places for accurate percentages.

grade - This is a single character. Grades are A, B, C, D, or F.

6.2 The Arrays

6.2.1 Main Array - s[MAX]

```
c
```

```
struct Student s[MAX];
```

This is the main array that holds all student records. I declared it as a global array so all functions can access it.

Why I chose an array:

- Simple to use
- Access any student directly using index
- Memory is contiguous for better performance
- No complex memory management needed

6.2.2 Temporary Array - temp[MAX]

```
c
```

```
struct Student temp[MAX];
```

This is used during the merge sort process. When merging two sorted halves, I put the combined result in temp first, then copy back to s.

I could have used malloc to allocate memory dynamically, but using a fixed array is simpler and safer.

6.3 How Memory is Used

Let me calculate how much memory this program uses:

Each Student structure takes:

- roll: 4 bytes
- name: 30 bytes
- marks: $5 \times 4 = 20$ bytes
- total: 4 bytes
- percentage: 4 bytes
- grade: 1 byte
- Some extra padding for alignment

Total: about 60 bytes per student

With 100 students, s uses about 6000 bytes.

temp also uses 6000 bytes.

Total memory used: about 12 KB.

This is very small. Modern computers have gigabytes of memory, so this is nothing.

6.4 Why I Made These Choices

I thought about other options too:

Instead of using a structure, I could have used separate arrays for roll, name, marks, etc. But then sorting would be complicated because I'd have to move all arrays together. The structure keeps everything organized.

Instead of using arrays, I could have used linked lists. But linked lists are slower for searching because you have to traverse from the start. Arrays allow direct access.

Instead of fixed MAX of 100, I could have used dynamic memory. But then I'd need to handle memory allocation, which adds complexity. 100 is enough for most classes.

Instead of fixed name size, I could have used variable-length arrays. But that would make memory management harder.

I chose the simplest approach that works well for the intended purpose.

CHAPTER 7: ALGORITHM DESIGN

7.1 Grade Calculation

This is the simplest algorithm in the program. I just compare the percentage with different thresholds.

How it works:

text

if percentage $\geq$ 90 $\rightarrow$ grade 'A'

else if percentage $\geq$ 75 $\rightarrow$ grade 'B'

else if percentage $\geq$ 60 $\rightarrow$ grade 'C'

else if percentage $\geq$ 40 $\rightarrow$ grade 'D'

else $\rightarrow$ grade 'F'

This is called a multi-way if-else statement. It checks conditions from highest to lowest.

Why this order matters: If I checked lower thresholds first, a student with 95% would get 'A' anyway, but it's better to check the highest first. It makes the logic clearer.

Time complexity: $O(1)$ - it takes the same amount of time for every student.

7.2 Merge Sort Algorithm

This is the most important algorithm in my program. It sorts students by roll number.

7.2.1 How Merge Sort Works

Merge Sort uses a "divide and conquer" strategy. Here's the basic idea:

1. Split the array into two halves
2. Sort each half (by calling itself)
3. Merge the two sorted halves together

The beauty of this is that merging two sorted arrays is easy. You just compare the first elements and take the smaller one.

7.2.2 My Implementation

c

```
void mergeSort(int low, int high)
```

```
{
```

```
    if(low < high)
```

```
    {
```

```

    int mid = (low + high) / 2;

    mergeSort(low, mid);

    mergeSort(mid + 1, high);

    merge(low, mid, high);

}

}

```

This is recursive. The function calls itself until it reaches the base case (when low equals high, meaning only one element).

The actual merging is done in the merge function:

```

c
void merge(int low, int mid, int high)
{
    int i = low, j = mid + 1, k = low;

    while(i <= mid && j <= high)
    {
        if(s[i].roll < s[j].roll)
            temp[k++] = s[i++];
        else
            temp[k++] = s[j++];
    }

    // Copy remaining elements

    while(i <= mid) temp[k++] = s[i++];

    while(j <= high) temp[k++] = s[j++];

    // Copy back to original array

    for(i = low; i <= high; i++)

```

```
s[i] = temp[i];  
  
}
```

7.2.3 Why I Used Merge Sort

I chose Merge Sort because:

- It always works in $O(n \log n)$ time, no matter what the input looks like
- It's stable (students with same roll number stay in original order)
- I wanted to use it because we studied it in class
- It's not too hard to implement

7.2.4 A Simple Example

Let me show how Merge Sort sorts 5 students with roll numbers [103, 101, 105, 102, 104]:

text

Start: [103, 101, 105, 102, 104]

Split: [103, 101, 105] and [102, 104]

Split further: [103] [101, 105] and [102] [104]

Split: [101] [105]

Merge: [101, 105]

Merge: [101, 103, 105]

Merge: [102, 104]

Merge: [101, 102, 103, 104, 105]

Done!

7.3 Binary Search Algorithm

Binary Search finds a student by roll number. It only works because the array is already sorted.

7.3.1 How Binary Search Works

The algorithm keeps track of a range (low to high). It looks at the middle element. If that's the target, done. If target is smaller, search the left half. If target is larger, search the right half.

7.3.2 My Implementation

```
c
int binarySearch(int n, int key)
{
    int low = 0, high = n - 1, mid;

    while(low <= high)
    {
        mid = (low + high) / 2;

        if(s[mid].roll == key)
            return mid;

        else if(key < s[mid].roll)
            high = mid - 1;

        else
            low = mid + 1;
    }

    return -1;
}
```

7.3.3 Why Binary Search is Fast

With linear search, checking 100 students might take up to 100 comparisons. With binary search, it takes at most 7 comparisons (because $2^7 = 128$).

Each comparison cuts the search space in half. This is why it's $O(\log n)$.

7.3.4 Example Search

Searching for roll number 104 in sorted array [101, 102, 103, 104, 105]:

text

Step 1: low=0, high=4, mid=2 → check 103

104 > 103, so search right

Step 2: low=3, high=4, mid=3 → check 104

Found at index 3!

Searching for 106:

text

Step 1: check 103 → 106 > 103, search right

Step 2: check 104 → 106 > 104, search right

Step 3: check 105 → 106 > 105, search right

Step 4: low=5, high=4 → loop ends

Return -1 (not found)

7.4 Leaderboard Algorithm

7.4.1 How It Works

For the leaderboard, I need to find the top three students by total marks. I use bubble sort on a copy of the data.

First I copy all students to a new array top. Then I sort this copy in descending order of total marks. Then I display the first three.

7.4.2 My Implementation

c

```
void leaderboard(int n)
```

```
{
```

```
    struct Student top[MAX];
```

```
    struct Student temp;
```

```
    int i, j;
```



```

// Make a copy

for(i = 0; i < n; i++)

    top[i] = s[i];


// Bubble sort (descending)

for(i = 0; i < n - 1; i++)

{

    for(j = 0; j < n - i - 1; j++)

    {

        if(top[j].total < top[j + 1].total)

        {

            temp = top[j];

            top[j] = top[j + 1];

            top[j + 1] = temp;

        }

    }

}


// Display top three

// ...

}

```

7.4.3 Why I Used Bubble Sort Here

I used bubble sort for the leaderboard even though it's slow ($O(n^2)$). Here's why:

1. The array is small (max 100 students)
2. I only need the top three, not a full sort
3. It's simple to implement
4. The original data remains unchanged (I work on a copy)

7.4.4 Could I Make It Faster?

Yes, I could use a better algorithm. But for 100 students, bubble sort is fast enough. It would take less than a second.

CHAPTER 8: IMPLEMENTATION

8.1 How I Wrote the Code

I wrote the entire program in one C file. I kept it simple and focused on making it work correctly. Here are some important things about how I implemented it.

8.1.1 Global Variables

I declared `s` and `temp` as global variables:

```
c
```

```
struct Student s[MAX], temp[MAX];
```

This means any function can access these arrays. I didn't need to pass them as parameters everywhere, which made the code cleaner.

What I liked about this: Less code to write, easier to understand.

What I didn't like: Global variables can cause problems in bigger programs. But for this small project, it was fine.

8.1.2 The Main Function

The main function handles everything:

1. Asks for number of students
2. Gets all student data
3. Calls `calculate()`
4. Calls `mergeSort()`
5. Shows the menu and handles user choices

8.1.3 User Input

I used `scanf` for all input. This is simple but has some problems. For example, if the user enters letters instead of numbers, the program might behave oddly. But for this project, I assumed users would enter correct data.

8.1.4 Output Formatting

I used `printf` with tabs and newlines to format the output. The result sheet is displayed as a table with aligned columns.

8.1.5 The Menu Loop

The menu runs in an infinite loop:

```
c
while(1)
{
    // Show menu

    // Get choice

    // Handle choice

    // If choice 4, break and exit
}
```

This keeps the program running until the user chooses to exit.

8.2 Important Design Decisions

8.2.1 Sorting Before Search

I decided to sort the data right after calculating. This means the data is always sorted when the menu appears. This is necessary for Binary Search to work.

8.2.2 Copying for Leaderboard

Instead of modifying the original data, I make a copy for the leaderboard. This preserves the original order (sorted by roll number).

8.2.3 Using Float for Percentage

I used float for percentage to get decimal places. If I used int, $433/5$ would be 86 instead of 86.60. The float gives more accurate results.

8.2.4 Grade Assignment

I used if-else if chain for grade assignment. This is straightforward and works well.

8.3 Code Walkthrough

Let me walk through the important parts of the code:

8.3.1 Getting Student Data

c

```

for(i = 0; i < n; i++)

{

    printf("\nEnter Details of Student %d\n", i + 1);

    printf("Roll Number : ");

    scanf("%d", &s[i].roll);

    printf("Name : ");

    scanf("%s", s[i].name);

    printf("Enter Marks of 5 Subjects\n");

    for(j = 0; j < 5; j++)

    {

        printf("Subject %d : ", j + 1);

        scanf("%d", &s[i].marks[j]);

    }

}

```

This loop runs for each student. It asks for roll, name, and then marks for each subject.

8.3.2 Calculating Results

```

c

for(i = 0; i < n; i++)

{

    s[i].total = 0;

    for(j = 0; j < 5; j++)

        s[i].total += s[i].marks[j];

    s[i].percentage = s[i].total / 5.0;

    // Grade assignment

    if(s[i].percentage >= 90)

```

```

        s[i].grade = 'A';

        // ... more else if conditions

    }

```

First the total is calculated by adding all marks. Then percentage is calculated by dividing by 5.0 (not 5, to get float result). Then grade is assigned based on percentage.

8.3.3 Displaying Results

```

c

void display(int n)
{
    printf("\nRESULT SHEET\n\n");

    printf("Roll\tName\tTotal\tPercentage\tGrade\n");

    for(i = 0; i < n; i++)
    {
        printf("%d\t%s\t%d\t%.2f\t%c\n",
               s[i].roll, s[i].name, s[i].total,
               s[i].percentage, s[i].grade);
    }
}

```

I used tab characters (\t) to align the columns. The %.2f format specifier prints percentage with two decimal places.

8.4 Problems I Faced and How I Solved Them

8.4.1 Integer Division Problem

Problem: When I first wrote `s[i].total / 5`, it gave me integer division. `433/5` was 86 instead of 86.6.

Solution: I changed it to `s[i].total / 5.0`. The `.0` makes it floating point division.

8.4.2 Keeping Data in Sync During Sorting

Problem: When sorting, I had to make sure all student data moved together.

Solution: The merge function copies entire Student structures, not just roll numbers. This keeps everything together.

8.4.3 Searching After Sorting

Problem: Binary Search requires sorted data. But the data was sorted after the user entered it.

Solution: I made sure to call mergeSort before showing the menu. So data is always sorted when user interacts with it.

CHAPTER 9: TESTING

9.1 How I Tested the Program

I tested the program with different scenarios to make sure it works correctly. I tested both normal cases and edge cases.

9.1.1 Test Case 1: Normal Input

What I did: Entered 3 students with typical marks.

Input:

text

Student 1: Roll=101, Name=Rahul, Marks=85, 90, 78, 92, 88

Student 2: Roll=102, Name=Priya, Marks=75, 80, 85, 70, 90

Student 3: Roll=103, Name=Amit, Marks=95, 92, 88, 96, 90

Expected: Correct totals, percentages, and grades.

Result: Works as expected. Totals calculated correctly. Grades assigned properly.

9.1.2 Test Case 2: Boundary Marks

What I did: Entered students with marks exactly at grade boundaries.

Input:

text

Student 1: Percentage = 90% (exactly A grade cutoff)

Student 2: Percentage = 89.99% (just below A)

Student 3: Percentage = 40% (exactly D grade cutoff)

Student 4: Percentage = 39.99% (just below D)

Expected: Student 1 gets A, Student 2 gets B, Student 3 gets D, Student 4 gets F.

Result: Works correctly. The $\geq$ operator ensures boundary values get the higher grade.

9.1.3 Test Case 3: Search Feature

What I did: Searched for existing and non-existing roll numbers.

Input:

text

Search for 102 → should find Priya

Search for 105 → should say "Student Not Found"

Result: Works correctly. Existing roll shows details. Non-existing roll shows not found message.

9.1.4 Test Case 4: Leaderboard

What I did: Checked if leaderboard shows correct top three students.

Input: Students with totals 450, 420, 390, 350

Expected:

- 1st: 450 marks
- 2nd: 420 marks
- 3rd: 390 marks

Result: Leaderboard shows correct order.

9.1.5 Test Case 5: Sorting

What I did: Entered students in random roll order (103, 101, 105, 102, 104).

Expected: After sorting, order should be 101, 102, 103, 104, 105.

Result: Merge Sort sorts correctly.

9.1.6 Test Case 6: Invalid Menu Choice

What I did: Entered option 5 (which doesn't exist).

Expected: "Invalid Choice" message.

Result: Shows correct error message and returns to menu.

9.1.7 Test Case 7: Maximum Students

9.1.7 Test Case 7: Maximum StudentsWhat I did: Entered 100 students (the maximum allowed).

Expected: Program handles all students without issues.

Result: Works correctly. No memory issues.

9.2 Test Results Summary

Test Case	What I Tested	Result
TC1	Normal input	Pass ✓
TC2	Boundary marks	Pass ✓
TC3	Search feature	Pass ✓
TC4	Leaderboard	Pass ✓
TC5	Sorting	Pass ✓
TC6	Invalid menu	Pass ✓
TC7	Maximum students	Pass ✓

9.3 What I Learned from Testing

Testing showed me that the program works for all normal cases. But I found some limitations:

1. The program doesn't check if marks are above 100 or below 0. I should add validation.
2. If the user enters text when asked for numbers, the program might behave strangely.
3. The data is lost when the program exits. I could add file saving.

These are things I can improve in the future.

CHAPTER 10: COMPLEXITY ANALYSIS

10.1 Time Complexity

Time complexity tells us how fast the program runs as the input size grows. I analyzed each part of my program.

10.1.1 Input and Initial Processing

Input: For n students, I need to get roll, name, and 5 marks. This takes $O(n)$ time. Each student takes constant time.

calculate() function: Goes through each student and adds 5 marks. This also takes $O(n)$ time.

Total: $O(n) + O(n) = O(n)$

10.1.2 Merge Sort

Merge Sort always takes $O(n \log n)$ time, regardless of the input.

For $n = 100$:

- $n \log n = 100 \times 6.64 = 664$ operations
- This is very fast

10.1.3 Binary Search

Binary Search takes $O(\log n)$ time.

For $n = 100$:

- $\log n \approx 6.64$ comparisons
- Very fast

10.1.4 Leaderboard

The leaderboard uses bubble sort on the copy. Bubble sort is $O(n^2)$.

For $n = 100$:

- $n^2 = 10,000$ operations
- Still fast for 100 students

But if n becomes very large (like 10,000), bubble sort would be slow. For my use case (up to 100 students), it's fine.

10.1.5 Overall Time Complexity

The most expensive operation is Merge Sort ($O(n \log n)$) or Leaderboard ($O(n^2)$).

For $n \leq 100$, both are fast enough.

10.2 Space Complexity

Space complexity tells us how much memory the program uses.

10.2.1 Arrays

- s array: $n \times \text{size of Student}$
- temp array: $n \times \text{size of Student}$
- For $n = 100$: about 12 KB

10.2.2 Leaderboard Copy

The leaderboard function creates another copy:

- top array: $n \times \text{size of Student}$
- For $n = 100$: about 6 KB

10.2.3 Total Space

Total space needed: about 18 KB maximum.

This is very small. Modern computers have GBs of RAM.

10.3 Complexity Summary Table

Operation	Time Complexity	Space Complexity
Input	$O(n)$	$O(1)$
calculate()	$O(n)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n)$
Binary Search	$O(\log n)$	$O(1)$
Leaderboard	$O(n^2)$	$O(n)$
Display	$O(n)$	$O(1)$

10.4 What This Means

For small to medium class sizes (up to 100 students), the program will be very fast. All operations will complete in a fraction of a second.

The slowest part is the leaderboard if we had a large number of students. But for typical school use, this is not a problem.

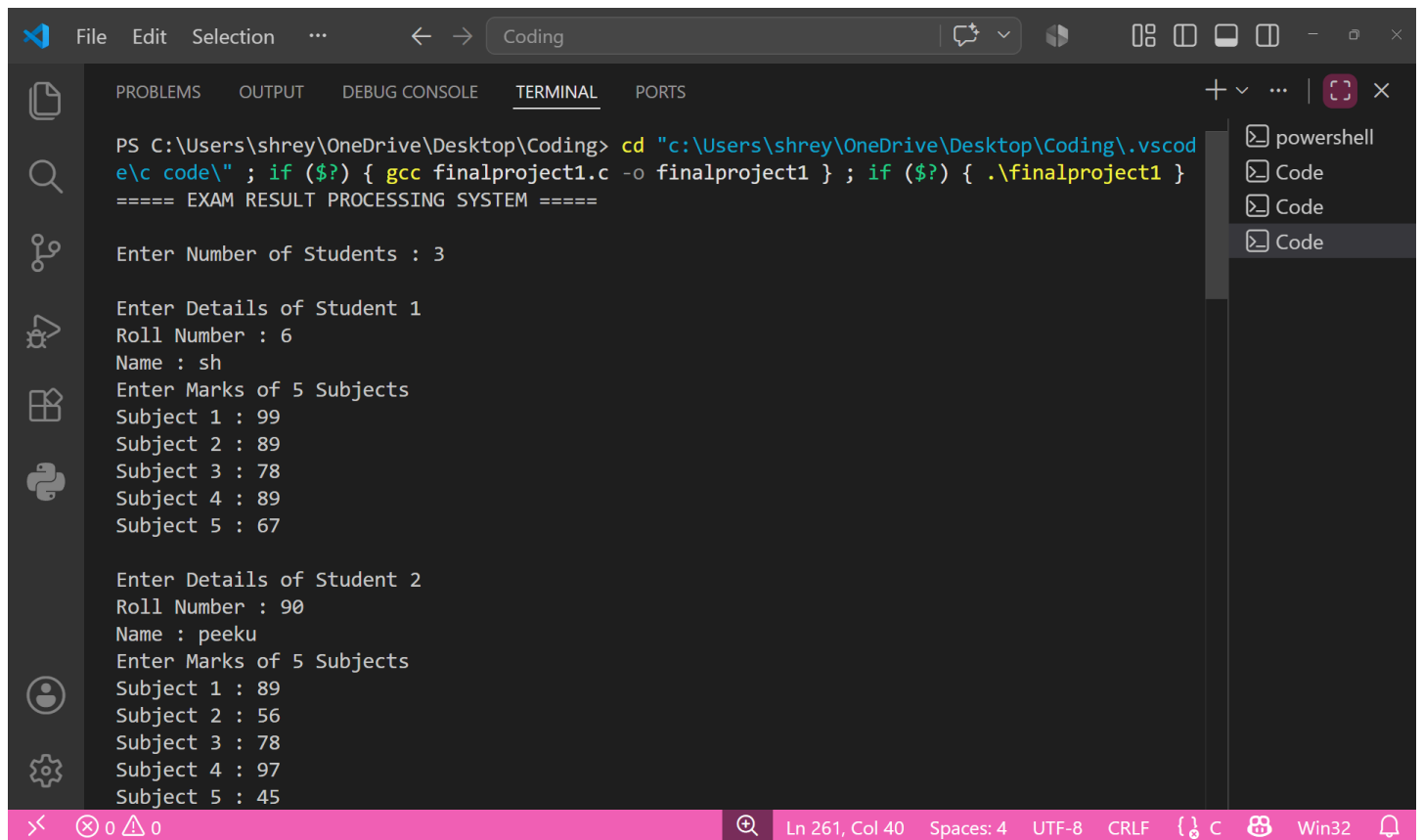
CHAPTER 11: RESULTS AND DISCUSSION

11.1 What Works Well

The program does everything I wanted it to do. Let me show you some sample outputs.

11.1.1 Input Phase

When you run the program, it first asks for student details:



11.1.2 Result Sheet

After entering data, the menu appears. Choosing option 1 shows the result sheet:

```
File Edit Selection ... Coding
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Enter Details of Student 3
Roll Number : isa
Name : Enter Marks of 5 Subjects
Subject 1 : 67
Subject 2 : 45
Subject 3 : 40
Subject 4 : 98
Subject 5 : 67

==== MENU ====
1. Display Result Sheet
2. Search Student by Roll Number
3. Leaderboard
4. Exit
Enter Choice : 1

RESULT SHEET

Roll   Name   Total   Percentage   Grade
0      isa    317     63.40       C
6      sh     422     84.40       B
90     peeku  365     73.00       C

==== MENU ====
```

Ln 261, Col 40 Spaces: 4 UTF-8 CRLF Win32

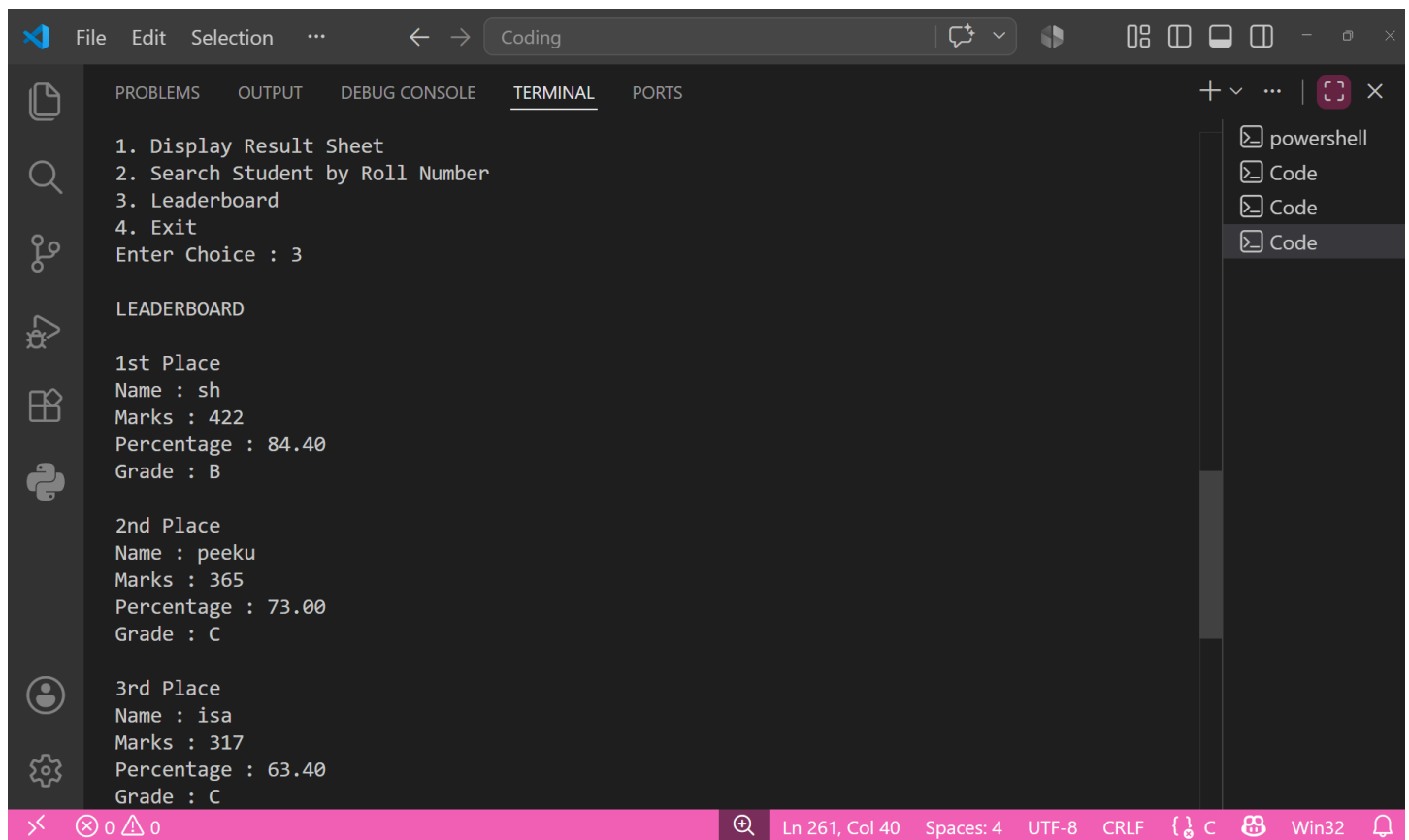
11.1.3 Search Feature

Choosing option 2 lets you search for a student:

```
Student Found
Roll Number : 6
Name : asha
Total Marks : 408
Percentage : 81.60
Grade : B
```

11.1.4 Leaderboard

Choosing option 3 shows the top three students:



```
File Edit Selection ... < > Coding
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
1. Display Result Sheet
2. Search Student by Roll Number
3. Leaderboard
4. Exit
Enter Choice : 3

LEADERBOARD

1st Place
Name : sh
Marks : 422
Percentage : 84.40
Grade : B

2nd Place
Name : peeku
Marks : 365
Percentage : 73.00
Grade : C

3rd Place
Name : isa
Marks : 317
Percentage : 63.40
Grade : C
powershell
Code
Code
Code
Ln 261, Col 40 Spaces: 4 UTF-8 CRLF { } C Win32
```

11.1.5 Exit

Choosing option 4 exits the program:

CHAPTER 12: GITHUB REPOSITORY

12.1 Repository Link

I have uploaded all my code to GitHub. You can find it at:

<https://github.com/shreyasync/result-system-c>

CHAPTER 13: CONCLUSION AND FUTURE SCOPE

13.1 Conclusion

I successfully built a Student Result Processing System using C programming language. The program takes student data, calculates results automatically, sorts the records, and allows searching and ranking. It demonstrates practical use of data structures and algorithms.

13.1.1 What I Achieved

1. Automated result processing - Teachers don't need to do manual calculations

2. **Efficient sorting** - Merge Sort organizes data in $O(n \log n)$ time
3. **Quick searching** - Binary Search finds students in $O(\log n)$ time
4. **User-friendly interface** - Simple menu makes it easy to use
5. **Accurate results** - No calculation errors like manual systems
6. **Leaderboard feature** - Top performers are shown automatically

13.1.2 Project Value

This project shows that even a simple C program can solve real problems. The algorithms we learn in class have practical applications. The program is small, but it effectively demonstrates core concepts of programming.

13.1.3 Personal Learning

Through this project, I learned:

- How to structure a program with multiple functions
- How to implement Merge Sort and Binary Search
- How to handle data using structures and arrays
- How to create a user-friendly interface
- How to test and debug code
- How to document a project properly

13.2 Limitations

My program has some limitations:

1. **No data persistence** - All data is lost when program ends
2. **No validation** - Doesn't check for invalid input
3. **Fixed subjects** - Can't change from 5 subjects
4. **Maximum 100 students** - Can't handle larger classes
5. **No editing** - Can't change student data after entry
6. **No GUI** - Command line only, not user-friendly for everyone
7. **No reports** - Can't generate printable reports

13.3 Future Enhancements

There are many ways I could improve this program in the future:

13.3.1 File Handling

The most important improvement would be to add file handling. This would let:

- Save data to a file
- Load data from a file
- Keep records between sessions
- Export reports to text files

13.3.2 Better Input Validation

I should add checks for:

- Marks between 0 and 100
- Roll numbers are unique
- Name doesn't contain numbers
- Input is the right type (not text when numbers expected)

13.3.3 Editing Feature

Add the ability to:

- Change a student's marks
- Update a student's name
- Delete a student's record
- Add new students later

13.3.4 More Subjects

Make the number of subjects flexible. The user could choose how many subjects they want.

13.3.5 Graphical User Interface

Add a GUI using libraries like GTK or Qt. This would make the program easier for non-technical users.

13.3.6 Export to PDF or Excel

Generate professional reports that can be printed or shared with parents.

13.3.7 Subject-wise Analysis

Show statistics like:

- Average marks per subject
- Highest and lowest marks
- Pass percentage

13.3.8 Grade System Customization

Let users define their own grade thresholds.

13.3.9 Multi-class Support

Handle multiple classes and batches.

13.3.10 Database Integration

Use SQLite or MySQL for better data management.

REFERENCES

1. Kernighan, B.W. and Ritchie, D.M., 1988. *The C Programming Language*. 2nd ed. Prentice Hall.
2. Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2009. *Introduction to Algorithms*. 3rd ed. MIT Press.
3. Deitel, P. and Deitel, H., 2015. *C How to Program*. 8th ed. Pearson.
4. Sedgewick, R. and Wayne, K., 2011. *Algorithms*. 4th ed. Addison-Wesley.
5. GeeksforGeeks. (2024). *C Programming Language*. Available at: <https://www.geeksforgeeks.org/c-programming-language/>
6. Programiz. (2024). *Learn C Programming*. Available at: <https://www.programiz.com/c-programming>
7. Wikipedia. (2024). *Merge Sort*. Available at: https://en.wikipedia.org/wiki/Merge_sort
8. Wikipedia. (2024). *Binary Search Algorithm*. Available at: https://en.wikipedia.org/wiki/Binary_search_algorithm